1 Code overview

`models.py`
Implements the five architectures required in Q 3: **Canonisation_Net**, **Symmetrization_Net**, **Sampled_Symmetrization_Net**, **Linear_eq_Net** (built from two equivariant layers), and **AugmentedInvariantNet**. Each class exposes `forward(X)` with $X \in \mathbb{R}^{N\times d}$ drawn from a synthetic white-Gaussian *matrix* generated by `torch.randn`.

`utils.py` • Permutation helpers: `create_permutations_sampled`.
- *Unit tests* for invariance/equivariance (`test_canonization_net`, ...).
- Training routine `train_variance_net` that learns variance purely from permutation augmentations.

`main.py`
Orchestrates the workflow:

1. runs each invariance test 50 times and prints the failure rate;
2. trains the variance model for 200 epochs on a fixed set of size $N = 50,\ d = 5$;
3. re-tests the trained model for invariance.

Dependencies are limited to `torch` and `numpy`; the script detects a GPU via `torch.cuda.is_available()` and moves tensors accordingly.

2 Permutation–invariance sanity check

test Permutation invariance

For every network we apply 50 random permutations $P \in S_N$ to a fixed set X and record a *failure* when the ℓ_∞–distance between the original and permuted outputs exceeds a model-specific tolerance:

(a,b,d) fail if $\|f(X) - f(PX)\|_\infty > 10^{-5}$;

(c) fail if $\|f(X) - f(PX)\|_\infty > 2 \times 10^{-1}$
(higher threshold to reflect the Monte-Carlo approximation of S_N);

(e) fail if $\|f(X) - f(PX)\|_\infty > 5 \times 10^{-2}$
(relaxed tolerance to probe the invariance learned via augmentation).

Finally, for each model we report the *failure rate*

$$\text{Failure rate} \;=\; \frac{\#\text{ failed permutations}}{\#\text{ permutations tested}}\,,$$

i.e. the fraction of the 50 sampled permutations that violate their respective tolerance.

Network	% non-invariant ↓	Pass?
Canonisation	0.00	✓
Full symmetrisation	0.00	✓
Sampled symmetrisation ($K = 256$)	0.94	✗
S_N-equivariant linear layers	0.00	✓
Learned-invariant (augmentations)	0.28	✓*

Table 1: Failure rate for each network.

Discussion

- **Exact symmetry methods.** Canonisation, full symmetrisation, and linear S_N-equivariant layers are provably invariant; failures are zero within numerical precision.
- **Monte-Carlo method (c).** Sampled symmetrisation averages over $K = 0.05 \cdot 8!$ permutations— $\approx 0.05\,|S_N|$ for $N = 8$ - so estimator variance might explain the $\approx 1\%$ failure rate. Increasing K should decrease the error.
- **Learned invariance (e).** The unconstrained MLP acquires symmetry through augmentation; after $\sim$ 250 epochs with $\sim$ 500 augmentations per epoch the failure rate falls below 0.3%.

* Both stochastic models (c and e) pass with a looser tolerance, We intentionally kept a looser threshold here to reveal the *trend* toward invariance and to assess given our time and computation resources how far each model has progressed toward full invariance.

2.1 Networks: theory & implementation details

(a) **Canonisation Net**

Theory. A canoniser g assigns every permutation orbit to a single representative. We have $f(g(x)x) = f(g(y)y)$ whenever x and y are in the same orbit thereby proving exact permutation invariance.

Implementation. We define the Canonization function g by sorting the rows according to *descending* Euclidean-norm order (computed over the feature dimension) with PyTorch sort (`idx = x.norm(dim=1).sort(descending=True).indices`). After re-ordering we apply a row encoder `Linear(d,64) → ReLU → Linear(64,64)` and feed it into the head `Linear(64,d_out)`.

(b) **Full symmetrisation**

Theory. The average $\bar{f}(X) = |S_N|^{-1} \sum_{P \in S_N} f(PX)$ is analytically invariant.

Implementation. For $N = 6$ we enumerate all permutations with `itertools.permutations`. Each permuted set passes through the same MLP (`Linear → ReLU → mean`), the results are accumulated and divided by $N!$.

(c) **Sampled symmetrisation**

Theory. We approximate the group average with $\hat{f}_K(X) = K^{-1} \sum_{k=1}^{K} f(P_k X)$, where P_k are i.i.d. uniform permutations.

Implementation. We sample $K = 0.05 \cdot N!$,$(N = 8)$ permutations with `torch.randperm(N)`. Inside the loop we apply the row MLP, *mean-pool*, and accumulate into a vector of shape $(d_{\text{out}},)$. Finally we divide by K.

(d) **Linear S_N-equivariant network**

Theory. Every *permutation-equivariant* linear map acting on an $N \times d_{\text{in}}$ tensor (see Lec. 10) must have the following form

$$(\Phi X)_i \;=\; XW_1 \;+\; 11^T XW_2.$$

Stacking such Φ's with preserves equivariance; a final row-sum makes the overall map invariant.

Implementation. The layer below realises Eq.((d))

Listing 1: Equivariant layer in PyTorch

```
class Linear_eq_layer(nn.Module):
def __init__(self, d_in=10, d_hidden=32):
super().__init__()
self.w1 = nn.Linear(d_in,d_hidden)
self.w2 = nn.Linear(d_in,d_hidden)


def forward(self, x):  # x is (n, d_in)
return self.w1(x) + self.w2(torch.unsqueeze(torch.sum(x,dim=0)
```

Equivariance test. property was verified separately.
Followed by two such layers, ReLU activations, row-sum pooling, and a head `Linear(d_h,d_out)`.

Listing 2: Stacked equivariant layers in PyTorch

```
class Linear_eq_Net(nn.Module):
def __init__(self, d_in=10, d_hidden=32 , d_out = 4):
super().__init__()
self.equiv = nn.Sequential(Linear_eq_layer(d_in,d_hidden),
nn.ReLU(),
Linear_eq_layer(d_hidden,d_hidden)
)

self.post_pool = nn.Sequential(
nn.ReLU(),
nn.Linear(d_hidden, d_out))


def forward(self, x):  # x is (n, d_in)
x_eq = self.equiv(x)
x_pool = x_eq.sum(dim=0)
return self.post_pool(x_pool)
```